

Genel değerlendirme

Kod tabanı küçük/orta ölçek için anlaşılır ve işlevsel; ancak sürüş kaydı büyüdükçe performans, veri tutarlılığı ve ekranın sorumluluk yoğunluğu belirgin teknik borç oluşturacak. En kritik alan canlı sürüş akışı.

Mevcut Mimari ve İyi Yanlar

- Feature-first ayırım mantıklı: **features/** ekranları, **services/** iş kuralları/veri erişimini, **core/** platform izinlerini, **theme/** görsel ortaklıkları topluyor.
- State management, mevcut kapsam için sade **StatefulWidget + setState + FutureBuilder** yaklaşımı. Ek bir paket olmadan okunabilir kalmış.
- **Future** nesneleri **initState** içinde saklanıyor; örneğin profil ve sürüş geçmişi her **build** çağrısında tekrar sorgulanmıyor.
- Canlı sürüş sekmesinin **Offstage** ile korunması, sekme değişince aktif harita/sürüş durumunun kaybolmasını önüyor:
[main_navigation_screen.dart](D:\\FlutterProjects\\gidon\\lib\\features\\navigation\\main_navigation_screen.dart).
- GPS, istatistik ve skor hesapları UI'dan ayrılmış. Özellikle
[ride_stats_calculator.dart](D:\\FlutterProjects\\gidon\\lib\\services\\location\\ride_stats_calculator.dart) ve
[score_calculator.dart](D:\\FlutterProjects\\gidon\\lib\\services\\scoring\\score_calculator.dart) saf hesap fonksiyonları içeriyor; test etmeye uygun.
- Isar modellerinde sürüş kimliği ve zaman alanlarının indekslenmesi doğru:
[gps_point.dart](D:\\FlutterProjects\\gidon\\lib\\services\\location\\gps_point.dart).
- Arka plan isolate'ı doğru entry-point ile başlatılıyor, kendi Isar bağlantısını açıyor ve foreground bildirimini yönetiyor:
[background_service.dart](D:\\FlutterProjects\\gidon\\lib\\services\\background\\background_service.dart).
- Controller/timer temizliği çoğu yerde iyi: canlı sürüş ekranı timer, konum servisi ve text controller'ı kapatıyor; harita ve menü animation controller'ları da dispose ediliyor.
- Null-safety genel olarak iyi uygulanmış: **required**, nullable alanlar ve **mounted** kontrolleri yaygın. Veri modellerindeki immutable sonuç sınıfları da yerinde.

Kötü Yanlar ve Teknik Borçlar

1. Canlı sürüş ekranı “god widget” olmaya başlamış

[LiveTrackingScreen](D:\\FlutterProjects\\gidon\\lib\\features\\live_tracking\\live_tracking_screen.dart) yaklaşık 700 satırda izin, GPS UI, servis komutları, Isar okuma, istatistik, skor, yükseklik

API'si, arama ve harita durumunu yönetiyor. Bu, yeni özellikler eklendikçe spaghetti koda dönüşme riski yüksek bir yapı.

Ekranın doğrudan `IsarService`, `FlutterBackgroundService`, `WakelockPlus`, geocoding ve elevation servislerini çağırması; iş akışını test etmeyi ve değiştirmeyi zorlaştırıyor.

2. Aynı iş alanında birbirini tekrar eden iki kayıt yaklaşımı var

[ride_session_service.dart](D:\\FlutterProjects\\gidon\\lib\\services\\location\\ride_session_service.dart) GPS dinleyip Isar'a yazan bir `RideSessionService` içeriyor, fakat uygulamanın gerçek akışı doğrudan

[background_service.dart](D:\\FlutterProjects\\gidon\\lib\\services\\background\\background_service.dart) üzerinden ilerliyor. Bu servis kullanılmıyor gibi görünüyor.

Bu, "hangi sınıf sürüş kaydının sahibi?" sorusunu belirsizleştiriyor. Tek bir `RideRecordingController/UseCase` altında birleştirilmesi gerekir.

3. Sürüş verisi ve profil ödülleri atomik değil

Kaydetme sırasında önce `RideMeta`, sonra `UserProfile` farklı transaction'larda güncelleniyor:

[ride_summary_screen.dart](D:\\FlutterProjects\\gidon\\lib\\features\\live_tracking\\ride_summary_screen.dart). Uygulama bu iki işlem arasında kapanır ya da ikinci işlem hata verirse, geçmişte görünen ama ödülü uygulanmamış bir sürüş oluşabilir.

Ayrıca "kaydedildi", "ödül uygulandı", "silindi" durumları modelde açıkça temsil edilmiyor; tamamlanmış sürüşün ölçütü `RideMeta` varlığı. Bu gizli durum modeli ileride veri onarımı ihtiyacı yaratır.

4. Silinen sürüş ile profil toplamaları bilinçli olarak ayırıyor

Geçmişten sürüş silinince GPS ve meta verisi siliniyor, ancak XP, mesafe, seviye ve toplamalar geri alınmıyor:

[ride_summary_screen.dart](D:\\FlutterProjects\\gidon\\lib\\features\\live_tracking\\ride_summary_screen.dart). Bu bir ürün kararı olabilir; ancak kullanıcı "sürüş silindi ama toplam kilometrem niye düşmedi?" diyecekse veri tutarlılığı problemi haline gelir.

En azından bu davranış kalıcı bir ürün kuralı olarak modellenmeli veya silme işlemi ödülleri/aggregate'leri tersine çevirmelidir.

5. Hata ve yarış durumu yönetimi eksik

- Başlat/durdur akışı yeniden girişe açık. Kullanıcı bekleyen `await` işlemleri sırasında tekrar dokunursa birden çok start/stop komutu oluşabilir.

- Durdurma komutu arka plan isolate'ına gönderildikten hemen sonra noktalar okunuyor: [live_tracking_screen.dart](D:\\FlutterProjects\\gidon\\lib\\features\\live_tracking\\live_tracking_screen.dart). Servisin son GPS yazımını bitirdiğine dair onay beklenmiyor.
- İptalde de stop komutundan hemen sonra noktalar siliniyor; arka plan isolate'ı son bir noktayı daha yazarsa silinen sürüş tekrar veri alabilir.
- Yer arama isteğinde hata yönetimi, timeout ve eski isteği geçersiz kılma yok: [geocoding_service.dart](D:\\FlutterProjects\\gidon\\lib\\features\\map\\geocoding_service.dart). Ağ hatasında **_isSearching** açık kalabilir; eski sonuç daha yeni sorguyu ezebilir.
- **FutureBuilder** ekranları çoğunlukla **snapshot.hasError** durumunu göstermiyor. Hata, boş liste veya sonsuz yükleme gibi görünür.

Performans ve Bellek Yönetimi

Kritik: her saniye tüm sürüş yeniden okunuyor ve hesaplanıyor

Aktif sürüşte timer her saniye çalışıyor:

[live_tracking_screen.dart](D:\\FlutterProjects\\gidon\\lib\\features\\live_tracking\\live_tracking_screen.dart). Her tikte:

1. O sürüşün tüm GPS noktaları Isar'dan çekiliyor.
2. Mesafe, ortalama hız, maksimum hız, ivme, yükseklik, hareket süresi ve XP ayrı ayrı hesaplanıyor.
3. Nokta listesi state'e yeniden atanıyor.
4. Harita ve rota tekrar çiziliyor.

Bu yaklaşım sürüş uzadıkça **O(n)** maliyetini her saniye tekrarlar. GPS iki saniyede bir kaydedildiği için uzun bir sürüşte hem CPU hem bellek hem de disk okuması hızla artar.

Kritik: rota her rebuild'de çok sayıda nesne üretiyor

[route_polyline_builder.dart](D:\\FlutterProjects\\gidon\\lib\\services\\location\\route_polyline_builder.dart), iki noktalı her segment için ayrı **Polyline** oluşturuyor.

[live_route_map_view.dart](D:\\FlutterProjects\\gidon\\lib\\features\\live_tracking\\live_route_map_view.dart) ise bu listeyi her build'de yeniden üretiyor.

Binlerce GPS noktası = binlerce **Polyline** ve her saniye yeni allocation demek. Uzun rotalarda jank üretmesi çok olası.

Kritik: geçmiş listesi tüm GPS geçmişini belleğe alıyor

[getAllRideOverviews](D:\\FlutterProjects\\gidon\\lib\\services\\location\\isar_service.dart) önce bütün **GpsPoint** kayıtlarını çekiyor, bellekte sürüşlere gruplayıp her sürüş için özet hesaplıyor. Ardından her sürüş için ayrı metadata sorgusu çalışıyor (**N+1 query**).

Bu, birkaç sürüşte kabul edilebilir; aylardır veri biriktiğinde profil/geçmiş açılışını pahalı hale getirir. Özet bilgiler **RideMeta** yerine ayrı bir **Ride** aggregate modelinde sürüş tamamlanırken saklanmalı; liste o tablodan sayfalı okunmalıdır.

Orta seviye noktalar

- Her GPS noktası ayrı Isar write transaction ile yazılıyor. İki saniye aralıkta çalışsa da uzun sürüşlerde disk/enerji maliyeti oluşturur. Kuyruklu ve kısa periyotlu batch yazım daha verimli olur.
- Canlı konum akışı UI tarafında sürüş başlamadan da açılıyor; sürüş sırasında arka plan servisinde ikinci bir GPS stream de açılıyor. Bu, konum sağlayıcı yükünü ve pil tüketimini artırabilir.
- [ProgressionResultScreen](D:\\FlutterProjects\\gidon\\lib\\features\\progression\\progression_result_screen.dart) gecikmeli animasyon dizisinde **mounted** kontrolü yapmadan **setState** çağırıyor. Ekran kapanırsa “setState after dispose” hatası oluşabilir.
- [RouteMapView](D:\\FlutterProjects\\gidon\\lib\\features\\live_tracking\\route_map_view.dart), giriş listesi boş değilken filtre sonrası listenin boş olabileceği durumu korumuyor. Tüm noktalar düşük doğrulukta ise bounds/ilk-son nokta erişiminde hata riski var.

Kod Kalitesi ve Clean Code

İyi olanlar:

- Kod dokümantasyonu ve alan adları genelde anlaşılır.
- **const** kullanımı UI içinde oldukça yaygın.
- Tipler açık; örneğin profil ekranındaki record tipi (**Future<(UserProfile, List<RideOverview>)>**) güvenli.
- **mounted** kontrolleri birçok **await** sonrası yerinde kullanılmış.

İyileştirilmesi gerekenler:

- **late final Isar isar, initialize()** çağrısı unutulduğunda geç çalışma zamanı hatası üretir. Uygulama başlangıcında doğru kullanılsa da, bağımlılık yönetimi bunu daha güvenli kılabilir.
- Callback için **async void** tarzındaki **_toggleRide()** hata yakalamayı zorlaştırır. **Future<void>** dönüp merkezi hata/işlem durumu yönetimi tercih edilmeli.
- Tema altyapısı tanımlı fakat global uygulama temasına bağlanmamış: [app_theme.dart](D:\\FlutterProjects\\gidon\\lib\\theme\\app_theme.dart),

[main.dart](D:\\FlutterProjects\\gidon\\lib\\main.dart). Ekranlar doğrudan **AppColors** kullanıyor; görsel kararlar dağılmış durumda.

- **RideSessionService**, **AppTheme** ve ayrı **RideHistoryScreen** gibi kullanılmayan/tekrarlı yapılar kod tabanının niyetini bulanıklaştırıyor.
- Test kapsamı pratikte yok. Tek test placeholder: [widget_test.dart](D:\\FlutterProjects\\gidon\\test\\widget_test.dart). Özellikle istatistik, XP, pause ve kayıt-silme transaction senaryoları test edilmelidir.
- Doğru yükseklik API'sinden gelen **elevationGainOverride**, **RideSummary.fromPoints** içinde yalnızca boş rota durumunda kullanılıyor; dolu rota için ham GPS irtifası yazılıyor: [ride_stats_calculator.dart](D:\\FlutterProjects\\gidon\\lib\\services\\location\\ride_stats_calculator.dart). Bu, XP hesabı ile ekranda/profilde görünen yükseklik değerlerinin farklılaşmasına yol açan somut bir doğruluk hatasıdır.

Öncelikli İyileştirme Listesi

1. **Sürüş akışını tek bir state sahibi altında çıkarın.**
LiveTrackingScreen içindeki iş kurallarını **RideRecordingController** veya Riverpod/Bloc tabanlı bir notifier'a taşıyın. Durumları açıkça modelleyin: **idle**, **starting**, **recording**, **stopping**, **reviewing**, **saving**, **failure**. Start/stop için arka plan servisinden acknowledgment bekleyin.
2. **Veri modelini sürüş özeti merkezli hale getirin.**
Ride kaydında durum, başlangıç/bitiş, özet metrikler, ödül durumu ve metadata bulunsun; **GpsPoint** yalnızca rota detayları olsun. Sürüş tamamlanınca özet ve profil ödülü tek kontrollü transaction/iş akışında yazılsın. Geçmiş ekranı tüm GPS noktalarını çekmek yerine bu özet kayıtlarını sayfalı sorgulasın.
3. **Canlı rota güncellemesini artımlı ve seyrek hale getirin.**
Her saniye tam DB okuması ve tüm rota hesaplaması yerine yeni GPS noktasını stream üzerinden state'e ekleyin; istatistikleri artımlı hesaplayın. Harita polyline'ını tek/az sayıda geometri ile oluşturun, noktaları sadeleştirin ve UI güncellemesini örneğin 2–5 saniyede bir throttle edin.

Bu üç adım, hem uygulamanın uzun sürüşlerde akıcılığını hem de yeni modlar (Ghost Map, leaderboard, senkronizasyon) eklenirken sürdürülebilirliğini en çok artıracaktır.